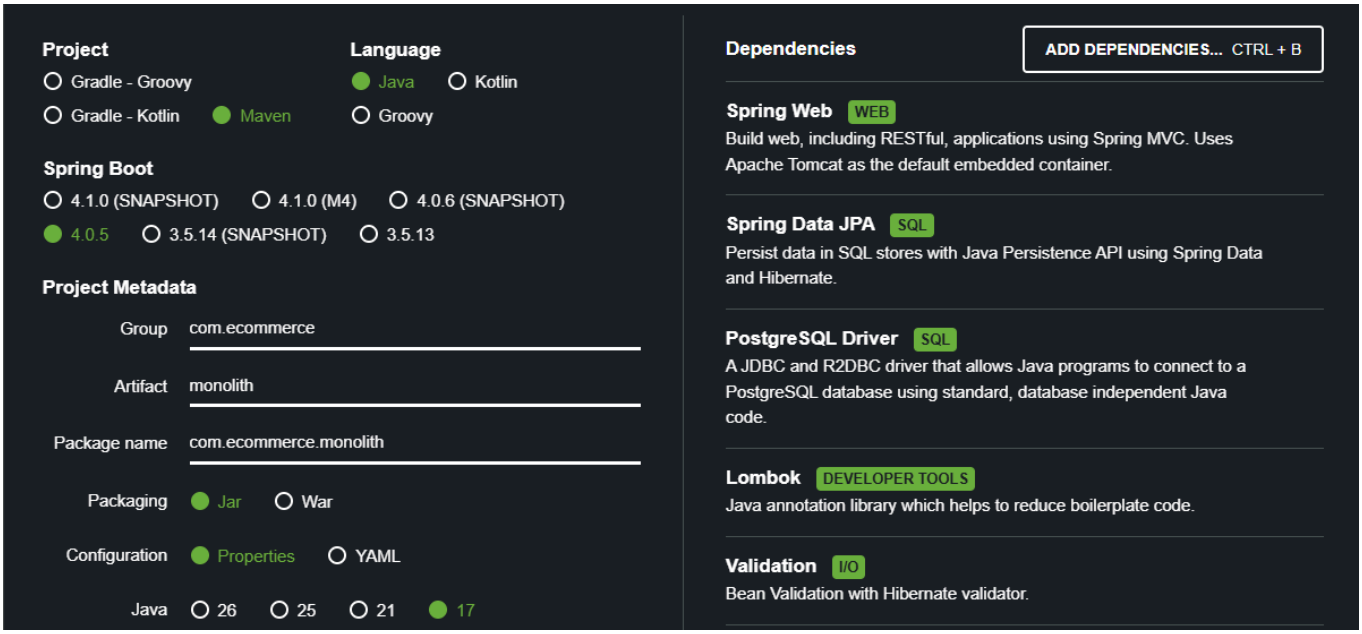
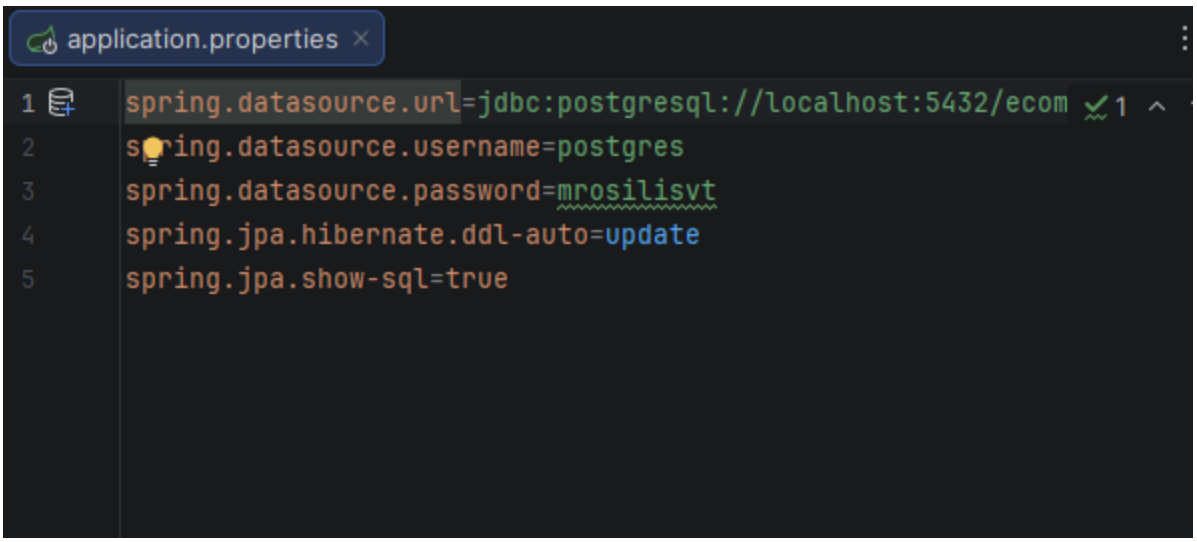
TP 1 - Application Monolithique avec Spring Boot
Gestion de Produits Simple
EL KHAYAR Youssef

Étape 1 : Initialisation du Projet:

J'ai créé le projet sur start.spring.io en sélectionnant Maven, Java 17, et les dépendances nécessaires (Spring Web, JPA, PostgreSQL, Lombok, Validation). Ensuite j'ai configuré le fichier application.properties pour connecter l'application à la base de données PostgreSQL locale.





Étape 2 : Créer l'Entité Product

J'ai créé la classe Product avec les annotations JPA pour la persistance et Lombok pour réduire le code répétitif..

```java
package com.ecommerce.monolith.product.model;
import ...
@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank(message = "Product name is required")
    @Column(nullable = false)
    private String name;
    @Column(length = 1000)
    private String description;
    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;
    @NotNull(message = "Stock is required")
    @PositiveOrZero(message = "Stock cannot be negative")
    @Column(nullable = false)
    private Integer stock;
}
```

Étape 3 : Créer le Repository

J'ai créé l'interface ProductRepository qui étend JpaRepository, ce qui nous donne automatiquement toutes les opérations CRUD sans écrire de requêtes SQL manuellement.

```java
package com.ecommerce.monolith.product.repository;

import ...

@Repository  4 usages
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByNameContainingIgnoreCase(String name);  no usages
}
```

Étape 4 : Créer le Service:

J'ai implémenté la couche service qui contient la logique métier, avec les méthodes getAll, getById, create, update et delete. Cette couche fait le lien entre le controller et le repository.

```
ication.properties   Product.java   ProductRepository.java   CategoryRepository.java   ProductService.java
1   package com.ecommerce.monolith.product.service;
2   import ...
8   @Service  3 usages  1 inheritor
9   @RequiredArgsConstructor
10  @Transactional
11  public class ProductService {
12      private final ProductRepository repository;
13      @Transactional(readOnly = true)  1 usage
14      public List<Product> getAll() { return repository.findAll(); }
17      @Transactional(readOnly = true)  3 usages
18      public Product getById(Long id) {
19          return repository.findById(id)
20                  .orElseThrow(() -> new IllegalArgumentException("Product not found with id: " + id));
21      }
22      public Product create(Product product) { return repository.save(product); }
25      public Product update(Long id, Product details) {  1 usage
26          Product product = getById(id);
27          product.setName(details.getName());
28          product.setDescription(details.getDescription());
29          product.setPrice(details.getPrice());
30          product.setStock(details.getStock());
31          return product; // save inutile si entité managed
32      }
33      public void delete(Long id) {  1 usage
34          Product product = getById(id);
35          repository.delete(product);
```

Étape 5 : Créer le Controller:

J'ai créé le ProductController qui expose les endpoints REST sur /api/products. Chaque méthode HTTP (GET, POST, PUT, DELETE) correspond à une opération CRUD et retourne les codes HTTP appropriés.

```java
package com.ecommerce.monolith.product.controller;
import ...
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
@Validated
public class ProductController {
    private final ProductService service;
    @GetMapping
    public ResponseEntity<List<Product>> getAll() {
        List<Product> products = service.getAll();
        return ResponseEntity.ok(products);
    }
    @GetMapping("/{id}")
    public ResponseEntity<Product> getById(@PathVariable Long id) {
        Product product = service.getById(id);
        return ResponseEntity.ok(product);
    }
    @PostMapping
    public ResponseEntity<Product> create(@Valid @RequestBody Product product) {
        Product created = service.create(product);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
    @PutMapping("/{id}")
    public ResponseEntity<Product> update(@PathVariable Long id,
                                          @Valid @RequestBody Product product) {
```


```java
                                          @Valid @RequestBody Product product) {
```


```java
        Product updated = service.update(id, product);
        return ResponseEntity.ok(updated);
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

Étape 6 : Tester avec Postman:

J'ai testé les 4 endpoints avec Postman et tout fonctionne correctement : POST retourne 201 Created, GET retourne 200 OK avec la liste des produits, PUT modifie bien le produit, et

DELETE retourne 204 No Content.

My Collection > New Request

Save | Share

POST http://localhost:8080/api/products Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "name": "Laptop",
3   "description": "Un bon laptop",
4   "price": 999.99,
5   "stock": 10
6 }
```

Body Cookies Headers 5 Test Results 201 Created 62 ms 249 B Save Response

{} JSON Preview Visualize

```
1 {
2     "id": 3,
3     "name": "Laptop",
4     "description": "Un bon laptop",
5     "price": 999.99,
6     "stock": 10
7 }
```

http://localhost:8080/api/products

Save | Share

GET http://localhost:8080/api/products Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cook

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results 200 OK 319 ms 408 B

{} JSON Preview Visualize

```
1 [
2     {
3         "id": 1,
4         "name": "Laptop",
5         "description": "Un bon laptop",
6         "price": 999.99,
7         "stock": 10
8     },
9     {
10        "id": 2,
11        "name": "Laptop",
12        "description": "Un bon laptop",
```

http://localhost:8080/api/products/1

Save | Share

PUT http://localhost:8080/api/products/1 Send

Docs | Params | Authorization | Headers 9 | Body | Scripts | Tests | Settings | Cooki

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers 5 | Test Results

200 OK · 135 ms · 254 B

{} JSON | Preview | Visualize


```
1 {
2     "id": 1,
3     "name": "Laptop Pro",
4     "description": "Version améliorée",
5     "price": 1299.99,
6     "stock": 5
7 }
```


http://localhost:8080/api/products/1

Save | Share

DELETE http://localhost:8080/api/products/1 Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers 3 | Test Results

204 No Content · 82 ms · 112 B

Raw | Preview | Visualize

```
1
```

Exercice:

J'ai créé l'entité Category et ajouté la relation ManyToOne dans Product via le champ category_id. En utilisant @RepositoryRestResource, Spring génère automatiquement les

endpoints pour les catégories. Les tests Postman confirment qu'on peut créer une catégorie et l'associer à un produit.

Créer l'entité Category:

```
ctRepository.java   CategoryRepository.java   ProductService.java   ProductController.java   Category.java

1  package com.ecommerce.monolith.product.model;
2
3  import jakarta.persistence.*;
4  import lombok.*;
5
6  @Entity
7  @Table(name = "categories")
8  @Getter
9  @Setter
10 @NoArgsConstructor
11 @AllArgsConstructor
12 @Builder
13 public class Category {
14
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
19     @Column(nullable = false)
20     private String name;
21 }
22
```

Modifier l'entité Product:

```java
package com.ecommerce.monolith.product.model;

import ...

@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank(message = "Product name is required")
    @Column(nullable = false)
    private String name;
    @Column(length = 1000)
    private String description;
    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;
    @NotNull(message = "Stock is required")
    @PositiveOrZero(message = "Stock cannot be negative")
    @Column(nullable = false)
    private Integer stock;
    @ManyToOne
    @JoinColumn(name = "category_id")
    private Category category;
}
```

Créer le Repository de Category:

```java
package com.ecommerce.monolith.product.repository;

import com.ecommerce.monolith.product.model.Category;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.rest.core.annotation.RepositoryRestResource;

@RepositoryRestResource(path = "categories")
public interface CategoryRepository extends JpaRepository<Category, Long> {
}
```

Tester dans Postman:

http://localhost:8080/categories

Save | Share

POST | http://localhost:8080/categories | Send

Docs | Params | Authorization | Headers 9 | Body | Scripts | Tests | Settings | Cookie

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers 9 | Test Results | 201 Created | 773 ms | 519 B

{} JSON | Preview | Visualize


```
{
    "name": "Électronique",
    "_links": {
        "self": {
            "href": "http://localhost:8080/categories/1"
        },
        "category": {
            "href": "http://localhost:8080/categories/1"
        }
    }
}
```


http://localhost:8080/api/products

Save | Sha

POST | http://localhost:8080/api/products | Sen

Docs | Params | Authorization | Headers 9 | Body | Scripts | Tests | Settings

Query Params

Key	Value	Description	Bulk E
Key	Value	Description	

Body | Cookies | Headers 5 | Test Results | 201 Created | 103 ms | 292 B

{} JSON | Preview | Visualize


```
{
    "id": 4,
    "name": "Laptop",
    "description": "Un bon laptop",
    "price": 999.99,
    "stock": 10,
    "category": {
        "id": 1,
        "name": "Électronique"
    }
}
```

http://localhost:8080/api/products

Save Share

GET http://localhost:8080/api/products Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cook

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results 200 OK 268 ms 483 B

{} JSON Preview Visualize


```
[
    {
        "id": 2,
        "name": "Laptop",
        "description": "Un bon laptop",
        "price": 999.99,
        "stock": 10,
        "category": null
    },
    {
        "id": 3,
        "name": "Laptop",
```


Globals Vault Tools

Résumé Ce TP m'a permis de construire une API REST complète avec Spring Boot en suivant l'architecture en couches Entity → Repository → Service → Controller. J'ai compris comment structurer un monolithe simple, appliquer les validations, et utiliser RestRepository pour simplifier le code. L'exercice sur Category m'a permis de mettre en pratique les relations JPA entre entités.